

用户级线程或内核级线程。用户级线程对操作系统是未知的，它们由一个在进程的用户空间中运行的线程库创建并管理。用户级线程非常高效，因为从一个线程切换到另一个线程不需要进行状态切换，但一个进程中一次只有一个用户级线程可以执行，如果一个线程发生阻塞，整个进程都会被阻塞。进程内包含的内核级线程是由内核维护的。由于内核认识它们，因而同一个进程中的多个线程可在多个处理器上并行执行，一个线程的阻塞不会阻塞整个进程，但从一个线程切换到另一个线程时需要进行模式转换。

4.10 推荐读物

[LEWI96]和[KLEI96]很好地综述了线程的概念，并介绍了相应的程序设计策略。前者侧重于概念，后者侧重于程序设计，但它们都涵盖了这两方面的主题。[PHAM96]深入讨论了 Windows NT 的线程机制。[ROBB04]全面讲述了 UNIX 的线程概念。

KLEI96 Kleiman, S.; Shah, D.; and Smallders, B. *Programming with Threads*. Upper Saddle River, NJ: Prentice Hall, 1996.

LEWI96 Lewis, B., and Berg, D. *Threads Primer*. Upper Saddle River, NJ: Prentice Hall, 1996.

PHAM96 Pham, T., and Garg, P. *Multithreaded Programming with Windows NT*. Upper Saddle River, NJ: Prentice Hall, 1996.

ROBB04 Robbins, K., and Robbins, S. *UNIX Systems Programming: Communication, Concurrency, and Threads*. Upper Saddle River, NJ: Prentice Hall, 2004.

4.11 关键术语、复习题和习题

4.11.1 关键术语

“套管”内核级线程	多线程	任务
轻量级进程	端口	线程
消息	进程	用户级线程

4.11.2 复习题

- 4.1 表 3.5 列出了无线程操作系统中进程控制块的基本元素。对于多线程系统，这些元素中的哪些可能属于线程控制块，哪些可能属于进程控制块？
- 4.2 请给出线程间的状态切换比进程间的状态切换开销更低的原因。
- 4.3 在进程概念中体现出的两个独立且无关的特点是什么？
- 4.4 给出在单用户多处理系统中使用线程的 4 个例子。
- 4.5 哪些资源通常被一个进程中的所有线程共享？
- 4.6 列出用户级线程相对于内核线程的三个优点。
- 4.7 列出用户级线程相对于内核线程的两个缺点。
- 4.8 定义“套管”技术。

4.11.3 习题

- 4.1 在进程中使用多线程有两个好处：（1）在进程中创建一个新线程的开销比创建一个新进程的开销小；（2）同一个进程内的线程间的通信更简单。同一个进程中两个线程切换的开销是否也比不同进程中两个线程切换的开销少？
- 4.2 在比较用户级线程和内核线程时曾指出用户级线程的一个缺点是，当一个用户级线程执行系统调用时，不仅这个线程被阻塞，进程中的所有线程都被阻塞。请问这是为什么？

4.3 OS/2 是 IBM 为个人计算机开发的一个过时的操作系统。在 OS/2 中，其他操作系统中通用的进程概念被分成了三个不同类型的实体：会话、进程和线程。一个会话是一组与用户接口（键盘、显示器、鼠标）相关联的一个或多个进程。会话代表了一个交互式的用户应用程序，如字处理程序或电子表格。这个概念使得 PC 用户可以打开一个以上的应用程序，在屏幕上显示一个或多个窗口。操作系统必须知道哪个窗口即哪个会话是活动的，以便把键盘和鼠标的输入传递给相应的会话。在任何时刻，只有一个会话在前台模式，其他的会话都在后台模式，键盘和鼠标的所有输入都发送给前台会话的一个进程。当一个会话在前台模式时，执行视频输出的进程直接把它发送到硬件视频缓冲区，然后发送到用户的屏幕；当这个会话移到后台时，硬件视频缓冲区被保存到一个逻辑视频缓冲区。当一个会话在后台时，如果该会话的任何一个进程的任何一个线程正在执行并产生屏幕输出，那么这个输出就被送到逻辑视频缓冲区；当这个会话返回前台时，屏幕会被更新，为新的前台会话显示逻辑视频缓冲区中的当前内容。

有一种方法可以把 OS/2 中与进程相关的概念的数量从 3 个减少到 2 个：删去会话，把用户接口（键盘、显示器、鼠标）和进程关联起来。这样，在某一时刻，就只有一个进程处于前台模式。为了进一步结构化，进程可分成几个线程。

a. 使用这种方法会丧失什么优点？

b. 若将这种修改方法深入下去，应该在哪里分配资源（内存、文件等），是在进程级还是在线程级？

4.4 考虑这样一个环境，用户级线程和内核级线程为一对一的映射关系，它允许进程中的一个或多个线程产生会引发阻塞的系统调用，而其他线程则继续运行。解释为什么在单处理器机器上，这个模型会使得多线程程序的运行速度快于相应单线程程序的运行速度。

4.5 当一个进程退出时，其正在运行的线程是否会继续运行？

4.6 OS/390 大型机操作系统围绕地址空间和任务的概念构造。粗略说来，一个地址空间对应于一个应用程序，并且或多或少地对应于其他操作系统中的一个进程；在一个地址空间中，可以产生一组任务，这些任务可以并发执行，这大致对应于多线程的概念。两个数据结构对管理任务结构起着关键作用。地址空间控制块（ASCB）含有 OS/390 所需要的关于一个地址空间的信息，而不论该地址空间是否正在执行。ASCB 中的信息包括分派优先级、分配给该地址空间的实存和虚存、该地址空间中就绪的任务数及每个任务是否被换出。一个任务控制块（TCB）表示一个正执行的用户程序，它含有在一个地址空间中管理该任务所需的信息，包括处理器状态信息、指向该任务所涉及的程序的指针和任务执行状态。ASCB 是在系统内存中保存的全局结构，而 TCB 是保存在各自地址空间中的局部结构。请问把控制信息划分成全局和局部两部分有什么好处？

4.7 在很多诸如 C 和 C++ 语言的现行规范中，并未提供对多线程编程的支持。如下面的程序所示，这一问题可能会对编译器和代码的正确性造成影响。考虑如下的函数定义和声明：

```
int global_positives = 0;
typedef struct list {
    struct list *next;
    double val;
} * list;
void count_positives(list l)
{
    list p;
    for (p = l; p; p = p -> next)
        if (p -> val > 0.0)
            ++global_positives;
}
```

考虑如下情况：当一个线程 A 执行操作

```
count_positives(<list containing only negative values>);
```

的同时，另一个线程 B 执行

```
++global_positives;
```

a. 该函数的功能是什么？

b. C 语言只对单个线程的执行进行了规范。对于本题中所声明的函数，在两个并发的线程中使用是

否会有明显或潜在的问题?

4.8 对于一些已优化的编译器(包括比较保守的 GCC), 上题中的 `count_positives` 函数一般会被优化成类似下面的代码:

```
void count_positives(list l)
{
    list p;
    register int r;
    r = global_positives;
    for (p = l; p; p = p -> next)
        if (p -> val > 0.0) ++r;
    global_positives = r;
}
```

如果有 A、B 两个线程并发执行, 被这样编译优化过的代码会有什么明显或潜在的问题发生?

4.9 考虑下列使用了 POSIX Pthread API 的代码:

```
thread2.c
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

int myglobal;

void *thread_function(void *arg) {
    int i, j;
    for (i=0; i<20; i++) {
        j=myglobal;
        j=j+1;
        printf(".");
        fflush(stdout);
        sleep(1);
        myglobal=j;
    }
    return NULL;
}

int main(void) {
    pthread_t mythread;
    int i;
    if ( pthread_create( &mythread, NULL, thread_function,
        NULL)) {
        printf("error creating thread.");
        abort();
    }

    for ( i=0; i<20; i++) {
        myglobal=myglobal+1;
        printf("o");
        fflush(stdout);
        sleep(1);
    }

    if ( pthread_join ( mythread, NULL ) ) {
        printf("error joining thread.");
        abort();
    }

    printf("\nmyglobal equals %d\n", myglobal);
    exit(0);
}
```

`main()` 中首先声明一个变量 `mythread`, 其类型为 `pthread_t`, 它是一个线程的 ID; 然后, `if` 语句创建一个与 `mythread` 关联的线程。调用 `pthread_create()`, 若成功则返回 0, 若失败则返回非零值。